unusual use of ActiveRecord features), but they don't affect your ability to test the features.

You do need to be careful if the database has added constraints that are not evident in the code. Typically this involves column constraints that go beyond any validations specified in the ActiveRecord model, or foreign key constraints that are not specified anywhere in the Rails code. Foreign key constraints are hardest to deal with. Older versions of Rails had no native mechanism for specifying them, but they are beloved by database admins the world over.

From a testing perspective, the problem is twofold. First there is business logic outside the Rails code and in the database. That logic is hard to find, test, and change. Even worse, foreign key constraints add dependencies that require certain objects to be created together. In a test environment, that kind of dependency leads to mysterious bugs: the database doesn't let you create test data, and objects that need to be created have nothing to do with the test but are there only to make the database happy. Keep a close eye on this in a legacy application created by a database-heavy development team that didn't trust ActiveRecord.

Using Test Doubles to Remove Dependencies

If you don't want to start off at 20,000 feet with acceptance tests, test doubles are another way to get tests started without disrupting the untested code.

In a legacy context, the advantage to using test doubles is their ability to isolate a single class and method from the rest of the application. When working with a legacy application, this allows you to temporarily put aside the issue of how shaky the rest of the application may be and focus on the single part you're trying to figure out at that very moment.

In practice this is similar to the test-double behavior-centric tests you saw in Chapter 7, *Using Test Doubles as Mocks and Stubs*, on page 129, except that the code already exists, so you don't have the element of designing the API.

Let's look at an example. Take the following legacy method (which we'll assume is part of some kind of nebulous order model):

```
def calculate_order_status
  self.total = 0
  line_items.each do |item|
    if item.quantity.blank?
      LineItem.delete(item.id)
      next
    end
    if item.cost.nil? then item.cost = 0 end
```